

C++ 프로그래밍 메모리 구조 레포트

학번/성명: 20230841/최준수

학부(과): 인공지능학부

□ 프로그램 실행 시 할당되는 메모리 영역(Code, Data, Stack, Heap)의 역할 조사

■ 코드 영역

- 실행할 프로그램의 기계어 코드(Instruction)가 저장되는 영역입니다.

■ 데이터 영역

- 프로그램의 전역(Global) 변수와 정적(Static) 변수가 저장되는 영역입니다.

■ 스택 영역

- 함수 호출과 관련된 지역(Local) 변수와 매개(Parameter) 변수가 저장되는 영역입니다.

■ 힙 영역

- 사용자가 직접 관리하는 동적 할당(Dynamic Allocation) 영역입니다.

□ 메모리 영역별 특성

■ 코드 영역

- CPU가 해당 코드를 해석하고 실행할 소스코드 그 자체가 저장됩니다. 해당 과정은 프로그램이 시작될 때 할당되고 종료될 때까지 유지됩니다.
- 실행 중 코드가 변경되는 것을 방지하기 위해 읽기 전용 상태로 보호됩니다.

■ 데이터 영역

- 프로그램 어디서든 접근 가능한 데이터를 저장합니다.
- 프로그램 시작과 동시에 할당되며, 프로그램이 종료되어야 메모리에서 해제됩니다.

■ 스택 영역

- 함수의 실행이 끝나면 사라지는 임시 데이터를 관리합니다.
- 선입선출 구조로 동작합니다.
- 컴파일 타임에 크기가 결정되며, 함수가 호출될 때 할당되고 복귀할 때 자동으로 해제됩니다.
- 메모리의 높은 주소에서 낮은 주소 방향으로 할당됩니다.

■ 힙 영역

- 개발자가 필요할 때 할당하고, 사용이 끝나면 반드시 직접 해제해야 합니다.
- 해제하지 않을 경우 메모리 누수가 발생할 수 있습니다.
- 메모리의 낮은 주소에서 높은 주소 방향으로 할당되며, 스택 영역과 공간을 공유하되 서로 반대 방향으로 커집니다.

□ Stack과 Heap이 메모리 공간을 공유하면서도 서로 반대 방향으로 확장되는 이유

■ 가용 공간의 유연한 활용

- 공간 효율성: 스택이 많이 필요한 프로그램이나, 힙(동적 할당)이 많이 필요한 프로그램 등 프로그램의 특성에 따라 남은 빈 공간을 유동적으로 나누어 쓸 수 있습니다.
- 충돌 지연: 양 끝에서 시작함으로써 두 영역이 서로 만나기(Stack-Heap Collision) 전까지 사용할 수 있는 최대 자유 공간을 확보할 수 있습니다.
- 한계 최적화: 한 영역을 고정 크기로 할당했을 때 발생하는 메모리 낭비를 줄이고, 시스템 자원을 한계치까지 효율적으로 활용하기 위함입니다.

□ 메모리 단편화(Fragmentation)의 원인은 무엇이고 해결 방안(Paging, Segmentation 등)에 대해 조사

■ 메모리 단편화의 정의 및 원인

- 메모리 공간이 작은 조각으로 나뉘어, 총 가용 공간은 충분함에도 불구하고 프로세스가 필요한 메모리를 할당받지 못하는 현상입니다.
- 크게 내부 단편화와 외부 단편화로 나뉘어 설명할 수 있습니다.

■ 내부 단편화

- 원인: 메모리 할당 시 프로세스가 필요한 양보다 더 큰 고정된 크기의 블록을 할당할 때 발생합니다.
- 결과: 할당된 블록 내부에서 사용되지 않는 빈 공간이 낭비됩니다.

■ 외부 단편화

- 원인: 프로세스의 생성과 소멸이 반복되면서, 메모리 사이사이에 사용하지 않는 작은 빈 공간들이 흩어질 때 발생합니다.
- 결과: 전체 빈 공간의 합은 프로세스 크기보다 크지만, 연속된 공간이 아니어서 할당이 불가능해집니다.

■ 메모리 관리 및 해결 기법

- 페이지: 물리 메모리를 프레임(Frame)이라는 고정된 크기로 나누고, 프로세스를 페이지(Page)라는 동일한 크기로 잘라 맵핑하는 방식입니다. 특징으로는 프로세스가 메모리에 불연속적으로 배치될 수 있어 외부 단편화가 발생하지 않습니다.
- 세그먼테이션: 메모리를 크기가 고정된 단위가 아니라, 코드(Code), 데이터(Data), 스택(Stack) 등 논리적 단위(Segment)로 나누어 할당하는 방식입니다. 특징으로는 각 세그먼트는 필요한 만큼의 크기만 할당받으므로 내부 단편화가 발생하지 않습니다.

□ 메모리 누수 (Memory Leak)과 이를 방지하기 위한 언어적 장치(Garbage Collection, Smart Pointer)를 비교 조사

■ 메모리 누수(Memory Leak)란

- 프로그램이 동적으로 할당한 메모리 영역(Heap)을 더 이상 사용하지 않음에도 불구하고, 시스템에 반환(해제)하지 않아 가용 메모리가 점점 줄어드는 현상입니다.
- 장시간 실행되는 프로그램의 경우 점진적으로 메모리 점유율이 높아져 시스템 성능 저하를 일으키고, 결국 프로세스가 강제 종료(Crash)될 수 있습니다.

■ 가비지 컬렉션 (Garbage Collection, GC)

- 대상 언어: Java, Python, C#, JavaScript 등
- 작동 원리: 시스템(런타임 엔진)이 백그라운드에서 더 이상 참조되지 않는 객체를 추적하여 자동으로 메모리를 해제합니다.
- 장점: 개발자가 메모리 해제에 신경 쓸 필요가 없어 생산성이 높고, 메모리 누수 위험이 현저히 낮습니다.
- 단점: GC가 동작하는 동안 프로그램이 일시적으로 멈추는 STW(Stop-The-World) 현상이 발생하여 실시간성이 중요한 시스템에서는 성능 저하의 원인이 됩니다.

■ 스마트 포인터 (Smart Pointer)

- 대상 언어: C++ (C++11 이후 표준)
- 작동 원리: 객체가 스코프(Scope)를 벗어날 때 소멸자가 자동으로 호출되는 원리를 이용해 메모리를 관리하는 RAII (Resource Acquisition Is Initialization) 패턴을 사용합니다.
- 주요 종류:
 - unique_ptr: 특정 객체에 대한 소유권을 독점하며, 복사가 불가능합니다.
 - shared_ptr: 참조 횟수를 관리하여, 해당 객체를 가리키는 포인터가 0이 될 때 자동으로 메모리를 해제합니다.
 - weak_ptr: shared_ptr의 순환 참조 문제를 해결하기 위해 사용됩니다.
- 장점: GC처럼 별도의 런타임 오버헤드가 거의 없으며, 메모리 해제 시점을 예측할 수 있어 성능 최적화에 유리합니다.
- 단점: 포인터의 소유권 개념을 명확히 이해하고 사용해야 하므로 학습 곡선이 존재합니다.